



De La Salle University

Department of Computer Technology

Laboratory for Computer Organization and Architecture

Laboratory Activity #01: Introduction to x86-64 Architecture Programming and SASM IDE

Name	Angel Arwen Reyes	Date Performed	September 10, 2026
------	-------------------	----------------	--------------------

I. Introduction

An application program is composed of machine instructions that are executed by a processor (or CPU). A CPU has a specific set of machine instructions also called instruction set architecture (ISA). The ISA of a CPU has a specific programming model that shows which registers can be used in the CPU, how data is loaded from or stored into memory and how data can be processed by the CPU. In this laboratory, we will be looking at the x86 architecture's ISA since it is one of the widely used CPUs in desktop and server computer systems.

The x86 architecture started from a 16-bit computer known as the Intel 8086 up to the 32-bit computer system with the Pentium III or Pentium 4 line. The x86 architecture that supports 64bit processing is called the x86-64 architecture (also called amd64) which are the later versions of Pentium 4 up to the current Intel Core i and Xeon series line of processors. The x86-64 architecture is backwards compatible with x86 architecture. The x86-64 software architecture model is shown Fig. 1.

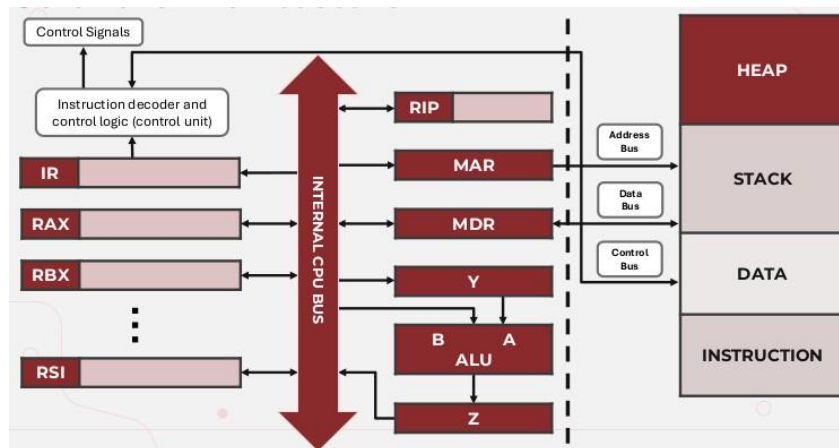


Fig. 1 x86-64 software architecture model

II. SASM Installation Configuration and Macro Tutorial

SASM is an Integrated Development Environment for x86 Assembly programming

1. Download and install SASM from its official website

(<https://dman95.github.io/SASM/english.html>)

For Windows, use the .exe installer

For Linux, use SASM that is available from the package manager (apt for debian / ubuntu based)

2. Configure SASM to use x86-64 Architecture.

SASM uses the x86 32bit architecture by default. 64bit can be configured by:

Settings > Build > x64 > Apply > ok

3. Create a new project

File > New

A new project will appear as shown in Fig. 2. Left portion of the window is taken up by the editor. Right portion for Input and Output. Bottom for Build Logs

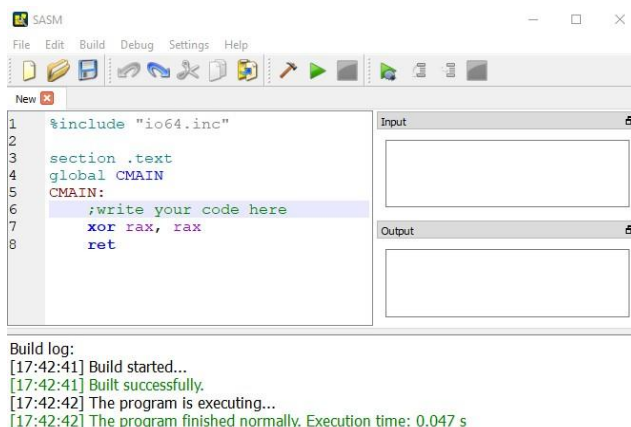


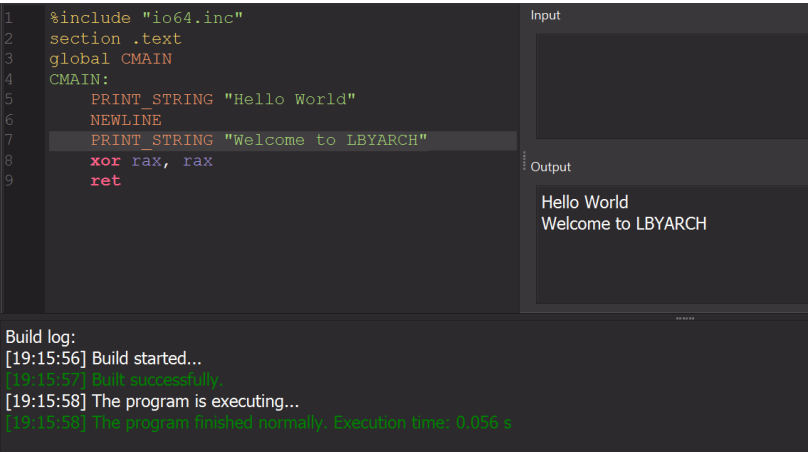
Fig. 2 SASM Project Window

Portions of the code will have already been automatically pre-written by the IDE. The library for Macros `io.inc` or `io64.inc` for 32 and 64 bit mode respectively. Section `.text` indicates the start of instruction code here declared as global is function `CMAIN`. The last two lines of code emulates `return 0` which indicates that the program exits successfully.

4. SASM macros enable access to the service calls provided as a macro, see appendix. These macros are a part of SASM and not the x86 architecture. Other IDE does not

have these macros. In place of the SASM macros for other IDE, Operating System service calls have to be implemented by hand.

5. PRINT_STRING macro enables printing of a string. For printing newlines, the PRINT_STRING macro does not support the \n escape character a NEWLINE macro has to be used instead. Code below prints "Hello World" and "Welcome to LBYARCH" in two separate Lines. Paste the code provided in SASM. Click build  then run  then provide a screenshot of a resized SASM window below which includes the editor, input, and Message Logs similar to Fig. 2

Code	Screenshot
<pre> #include "io64.inc" section .text global CMAIN CMAIN: PRINT_STRING "Hello World" NEWLINE PRINT_STRING "Welcome to LBYARCH" xor rax, rax ret </pre>	 Build log: [19:15:56] Build started... [19:15:57] Built successfully. [19:15:58] The program is executing... [19:15:58] The program finished normally. Execution time: 0.056 s

6. PRINT_CHAR are used for printing character use the code below and provide your observations regarding the PRINT CHAR macro

Code	Observations
<pre> PRINT_CHAR 'A' NEWLINE PRINT_CHAR 65 NEWLINE PRINT_CHAR 64+1 NEWLINE PRINT_CHAR 'ABC' </pre>	PRINT_CHAR only prints a single character and prints the character equivalent to its ASCII value. If more than one character is set to be printed, only the first character is printed.

7. PRINT_HEX function is used for displaying internal values in hexadecimal form use the code below and provide your observations regarding the PRINT_HEX macro

Code	Observations
<pre> PRINT_HEX 8, 10b NEWLINE PRINT_HEX 8, 10o NEWLINE PRINT_HEX 8, 10d NEWLINE PRINT_HEX 8, 10 </pre>	PRINT_HEX can take on a suffix appended next to a digit, like (b, o, d, h) to represent binary, octal, hex, and decimal input. The value is then converted to its equivalent hexadecimal representation before being printed.
<pre> NEWLINE PRINT_HEX 8, 10h NEWLINE PRINT_HEX 8, 0x10 NEWLINE PRINT_HEX 8, 10b+10o+10d+10h </pre>	0x has the same functionality as appending h to a digit. Thus, 10h and 0x10 both represent 16 in decimal. PRINT_HEX can also perform arithmetic using values from different number systems and print the resulting value in hexadecimal.

8. PRINT_DEC function is used for displaying internal values in signed decimal form use the code below and provide your observations regarding the PRINT_DEC macro.

Code	Observations
<pre> PRINT_DEC 8, 123 NEWLINE PRINT_DEC 8, -123 NEWLINE PRINT_DEC 8, 10+20-30*40/50 </pre>	PRINT_DEC prints values in signed decimal form, including negative values and the results of arithmetic expressions.

9. The first operand of the macro defines the size of data in **Bytes** to be displayed. The size of data in **bits** together with its type, here signed or unsigned, determines the values that it can validly represent. PRINT function's DEC and UDEC variants are for signed and unsigned. x86-64 uses 2's complement for representing signed values. Note: 1 Byte is 8 bits

Code	Observations
<pre> PRINT_DEC 2, 32767 NEWLINE PRINT_DEC 2, 32768 NEWLINE PRINT_UDEC 2, 65535 NEWLINE PRINT_UDEC 2, 65536 </pre>	The first two statements use PRINT_DEC with a size of 2 bytes (16 bits), giving a signed integer range of -32,768 to 32,767. Thus, 32767 is displayed correctly, while 32768 is interpreted as -32,768 because

	the highest bit is treated as the sign bit in two's complement representation. The next two statements use PRINT_UDEC, giving an unsigned integer range of 0 to 65535. In the output, 65,535 is displayed correctly, while 65,536 exceeds the range and wraps around to 0.
--	--

Equations for calculating the range of values that can be represented using signed and unsigned as follows:

Unsigned range: 0 to $2^n - 1$

Signed range: -2^{n-1} to $2^{n-1} - 1$

Where n is the number of **bits**

Complete the table below:

Bytes	bits	Unsigned Range	Signed Range
1	8	0 to 255	-128 to 127
2	16	0 to 65535	-32768 to 32767
4	32	0 to 4294967295	-2147483648 to 2147483647
8	64	0 to 1.84×10^{19}	-9.22×10^{18} to 9.22×10^{18}

What quirks did you observe below? _____

Code	Observations
<pre> PRINT_DEC 1, 127 NEWLINE PRINT_DEC 1, 128 NEWLINE PRINT_UDEC 1, 255 NEWLINE PRINT_UDEC 1, 256 </pre>	Although the size is specified as 1 byte (8 bits), PRINT_DEC and PRINT_UDEC can still display values outside the expected 8-bit range. For example, PRINT_DEC 1, 128 displays 128 instead of -128, while

	PRINT_UDEC 1, 256 displays 256 instead of 0.
--	--

Appendix

Table 1 - SASM Macros (<https://dman95.github.io/SASM/english.html>)

Macro name	Description
PRINT_UDEC <i>size, data</i> PRINT_DEC <i>size, data</i>	Print number <i>data</i> in decimal representation. <i>size</i> – number, giving size of <i>data</i> in bytes - 1, 2, 4 or 8 (x64). <i>data</i> must be number or symbol constant, name of variable, register or address expression without size qualifier (byte[], etc.). PRINT_UDEC print number as unsigned, PRINT_DEC — as signed.
PRINT_HEX <i>size, data</i>	Similarly previous, but data is printed in hexadecimal representation.
PRINT_CHAR <i>ch</i>	Print symbol <i>ch</i> . <i>ch</i> - number or symbol constant, name of variable, register or address expression without size qualifier (byte[], etc.).
PRINT_STRING <i>data</i>	Print null-terminated text string. <i>data</i> - string constant, name of variable or address expression without size qualifier (byte[], etc.).
NEWLINE	Print newline ('\n').